

Имитационное моделирование

Лабораторная работа №7. Дискретно-событийное моделирование

АВТОР
Гашимов Кенан Мухтар оглы

ПРИНАДЛЕЖНОСТЬ
Российский университет дружбы народов

1 Сведения об авторе

- ФИО: Гашимов Кенан Мухтар оглы
- Группа: НКНбд-01-23
- Студенческий билет: 1032235820
- Направление: Математика и компьютерные науки

2 Цель работы

Изучить дискретно-событийное моделирование на Julia, реализовать модели `M/M/c` и Росса, выполнить базовые и параметрические эксперименты, сохранить результаты в таблицы и графики, а также получить производные форматы из literate-кода: `md`, `ipynb`, `clean`.

3 Задание

1. Создать рабочий каталог проекта в структуре `DrWatson`.
2. Установить зависимости для дискретно-событийного моделирования, обработки таблиц, визуализации и литературного программирования.
3. Реализовать модель `M/M/c`.
4. Реализовать модель Росса с несколькими ремонтниками.
5. Выполнить базовые прогоны обеих моделей.
6. Выполнить параметрические исследования.
7. Построить графики динамики, загрузки и сравнений с аналитическими оценками.
8. Сохранить результаты в `CSV`.
9. Для каждого literate-скрипта получить `md`, `ipynb` и `clean`-представления.
10. Интегрировать результаты в отчёт и презентацию.

4 Теоретическое введение

4.1 Дискретно-событийное моделирование

Дискретно-событийное моделирование применяется для систем, состояние которых изменяется не непрерывно в каждый момент времени, а только в моменты наступления отдельных событий. Такими событиями в системах массового обслуживания являются поступление заявки, начало обслуживания и завершение обслуживания. В надёжных моделях событиями являются отказ оборудования, начало ремонта, окончание ремонта и падение системы. Такой подход позволяет не пересчитывать состояние системы на равномерной временной сетке, а переходить от одного события к следующему [1].

В лабораторной работе использовались две дискретно-событийные постановки: очередь `M/M/c` и модель Росса. В обоих случаях результат работы модели фиксировался в журналах событий, после чего по этим журналам вычислялись метрики и строились графики. Для воспроизводимости были заданы фиксированные начальные значения генераторов случайных чисел.

4.2 Модель `M/M/c`

Модель `M/M/c` относится к классическим моделям массового обслуживания. Первая буква `M` означает марковский, то есть пуассоновский входной поток с экспоненциальными интервалами между поступлениями. Вторая буква `M` означает экспоненциальное время обслуживания. Параметр `c` задаёт число параллельных одинаковых каналов обслуживания [2].

В работе использовались параметры:

Таблица 1: Параметры модели `M/M/c`

Параметр	Смысл
<code>lambda</code>	интенсивность входящего потока
<code>mu</code>	интенсивность обслуживания одного канала

Параметр	Смысл
<code>c</code>	число каналов обслуживания
<code>rho</code>	загрузка одного канала

Для базового сценария использованы значения `lambda = 0.9`, `mu = 0.5`, `c = 2`. При этих параметрах загрузка равна `rho = 0.9`, поэтому система остаётся устойчивой, но работает в режиме высокой нагрузки. Это хорошо подходит для демонстрации роста очереди и сравнения аналитических и имитационных оценок.

В качестве основных метрик использовались:

Таблица 2: Основные метрики модели `M/M/c`

Метрика	Смысл
<code>Wq</code>	среднее время ожидания в очереди
<code>W</code>	среднее время пребывания заявки в системе
<code>Lq</code>	среднее число заявок в очереди
<code>L</code>	среднее число заявок в системе
<code>Pwait</code>	вероятность того, что заявка будет ждать
<code>utilization</code>	загрузка каналов обслуживания

Имитационная модель генерирует заявки, назначает каждой заявке первый освободившийся канал и сохраняет для каждой заявки время прибытия, начала обслуживания и выхода из системы. На основе этих данных строится журнал событий и рассчитываются средние значения по заявкам и по времени.

4.3 Модель Росса

Модель Росса описывает систему с резервированием и ремонтом [3]. В системе постоянно должны работать `N` машин. Дополнительно есть `S` резервных машин. Если работающая машина выходит из строя, её заменяет резервная машина, а неисправная отправляется в ремонт. Если отказ произошёл в момент, когда резерва нет, система считается упавшей.

В работе использовались параметры:

Таблица 3: Параметры модели Росса

Параметр	Смысл
<code>N</code>	число постоянно работающих машин
<code>S</code>	число резервных машин
<code>repairers</code>	число ремонтников
<code>mean_time_to_failure</code>	средняя наработка машины до отказа
<code>mean_repair_time</code>	среднее время ремонта
<code>runs</code>	число независимых прогонов

Для модели Росса важны не только среднее время до падения, но и состояние ремонтной подсистемы. Поэтому дополнительно вычислялись средняя длина очереди на ремонт и загрузка ремонтников. Эти величины считались как средние по времени, а не как обычные средние по строкам журнала событий. Такой способ корректен для ступенчатых траекторий дискретно-событийной модели.

Аналитическое сравнение выполнялось через марковскую модель по числу доступных резервных машин. Для каждого состояния записывалось уравнение для среднего времени до поглощения, после чего система линейных уравнений решалась численно.

4.4 Инструменты реализации

Расчёты выполнены на языке Julia [4]. Структура проекта организована с помощью `DrWatson`, который предназначен для воспроизводимых вычислительных проектов и единообразного управления путями к данным, графикам и исходным файлам [5]. Таблицы сохранялись через `CSV.jl` и `DataFrames.jl`, графики строились с помощью `Plots.jl`.

4.5 Литературное программирование

Литературное программирование рассматривает программу как документ, в котором код и поясняющий текст образуют единый воспроизводимый материал. Такой подход удобен в лабораторной работе, потому что один исходный literate-скрипт может быть преобразован сразу в несколько форматов: исполняемый чистый скрипт, Markdown-документацию и Jupyter notebook [6].

В данной работе literate-версии использовались только для генерации производных форматов. Сами вычисления запускались обычными скриптами:

Таблица 4: Соответствие обычных и literate-скриптов

Обычный скрипт	Literate-версия	Производные форматы
<code>mmc.jl</code>	<code>mmc_literate.jl</code>	<code>mmc.md</code> , <code>mmc_clean.jl</code> , <code>mmc.ipynb</code>
<code>mmc_parameters.jl</code>	<code>mmc_parameters_literate.jl</code>	<code>mmc_parameters.md</code> , <code>mmc_parameters_clean.jl</code> , <code>mmc_parameters.ipynb</code>
<code>ross.jl</code>	<code>ross_literate.jl</code>	<code>ross.md</code> , <code>ross_clean.jl</code> , <code>ross.ipynb</code>
<code>ross_parameters.jl</code>	<code>ross_parameters_literate.jl</code>	<code>ross_parameters.md</code> , <code>ross_parameters_clean.jl</code> , <code>ross_parameters.ipynb</code>

Для генерации использовался пакет `Literate.jl` [7]. Метод `Literate.markdown` создавал Markdown-документ, `Literate.script` создавал чистую Julia-версию, а `Literate.notebook` создавал выполненный Jupyter notebook.

5 Выполнение лабораторной работы

5.1 Подготовка окружения

Работа началась с запуска Julia REPL.

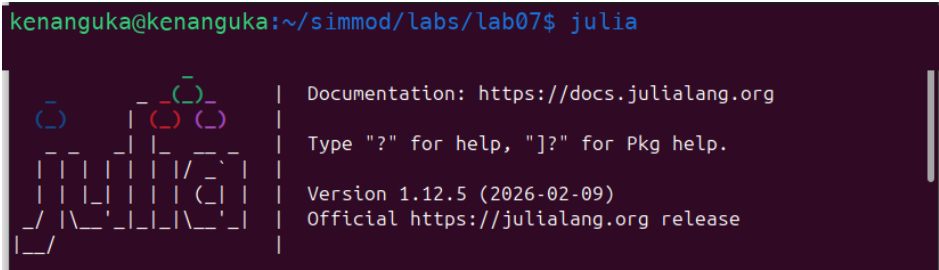


Рисунок 1: Запуск Julia

Далее был подключён пакет `DrWatson`.



Рисунок 2: Подключение DrWatson

Проект был создан в каталоге `lab_07_des`.

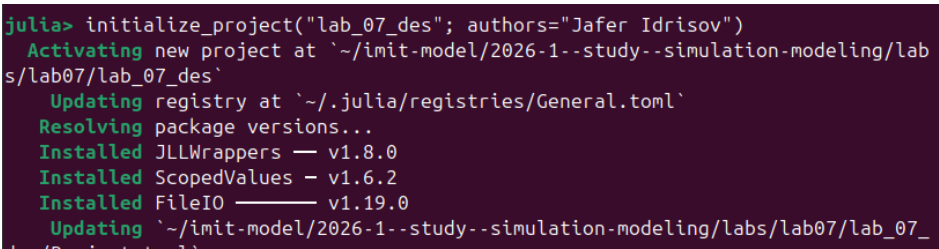


Рисунок 3: Инициализация проекта

После создания проекта были установлены зависимости.

```
julia> using Pkg

julia> Pkg.add([
    "DrWatson",
    "ConcurrentSim",
    "ResumableFunctions",
    "Distributions",
    "StableRNGs",
    "Random",
    "DataFrames",
    "CSV",
    "Statistics",
    "LinearAlgebra",
    "Plots",
    "StatsPlots",
    "Literate",
    "IJulia",
])
```

Рисунок 4: Установка пакетов

На следующем скриншоте показано завершение установки и предварительной компиляции пакетов.

```
Info Packages marked with ⚠ have new versions available but compatibility constraints restrict them from upgrading. To see why use `status --outdated -m`

Precompiling packages finished.
60 dependencies successfully precompiled in 259 seconds. 230 already precompiled.

julia> 
```

Рисунок 5: Завершение установки пакетов

5.2 Структура проекта

В проекте были подготовлены основные файлы:

- `src/QueueingModels.jl` — модуль с функциями моделей, расчёта метрик, сохранения таблиц и построения графиков;
- `scripts/mmc.jl` — базовый прогон `M/M/c`;
- `scripts/mmc_parameters.jl` — параметрическое исследование `M/M/c`;
- `scripts/ross.jl` — базовый прогон модели Росса;
- `scripts/ross_parameters.jl` — параметрическое исследование модели Росса;
- `scripts/*_literate.jl` — `literate`-версии сценариев;
- `scripts/*_clean.jl`, `docs/*.md`, `notebooks/*.ipynb` — производные форматы;
- `data/*.csv` — таблицы результатов;
- `plots/*.png` — графики.

5.3 Ключевые листинги кода

5.3.1 Модуль `QueueingModels.jl`

В модуле экспортированы функции для расчёта аналитики, запуска имитации и проведения параметрических исследований.

```
export mmc_analytic,
    simulate_mmc,
    run_mmc_experiment,
    run_mmc_parameter_scan,
    ross_analytic_crash_time,
    simulate_ross_once,
    run_ross_experiment,
    run_ross_parameter_scan
```

5.3.2 Аналитические метрики `M/M/c`

Функция `mmc_analytic` рассчитывает загрузку, вероятность ожидания и основные характеристики очереди.

```
function mmc_analytic(lambda::Real, mu::Real, c::Integer)
    rho = lambda / (c * mu)
    if rho >= 1
        return (
            rho = Float64(rho),
```

```

        p0 = NaN,
        pwait = NaN,
        lq = Inf,
        wq = Inf,
        w = Inf,
        l = Inf,
    )
end

a = c * rho
finite_sum = sum(a^n / factorial(n) for n in 0:(c - 1))
tail = a^c / (factorial(c) * (1 - rho))
p0 = 1 / (finite_sum + tail)
pwait = tail * p0
lq = rho / (1 - rho) * pwait
wq = lq / lambda
w = wq + 1 / mu
l = lambda * w
end

```

5.3.3 Имитация M/M/c

В имитации генерируются моменты прибытия, выбирается первый освободившийся канал, после чего фиксируются ожидание, обслуживание и время выхода.

```

for id in 1:num_customers
    arrival_time += rand(rng, interarrival_dist)
    server_id = findmin(server_available)[2]
    service_start = max(arrival_time, server_available[server_id])
    service_time = rand(rng, service_dist)
    departure_time = service_start + service_time
    server_available[server_id] = departure_time

    push!(rows, (
        id = id,
        arrival_time = arrival_time,
        service_start = service_start,
        departure_time = departure_time,
        wait_time = service_start - arrival_time,
        service_time = service_time,
        system_time = departure_time - arrival_time,
        server_id = server_id,
    ))
end

```

5.3.4 Запуск базового сценария M/M/c

```

using DrWatson
@quickactivate "lab_07_des"

include(sourcedir("QueueingModels.jl"))

QueueingModels.run_mmc_experiment(;
    lambda = 0.9,
    mu = 0.5,
    c = 2,
    num_customers = 5000,
    seed = 123,
)

```

5.3.5 Параметрическое исследование M/M/c

```

QueueingModels.run_mmc_parameter_scan(;
    lambdas = [0.3, 0.6, 0.9],
    channels = 1:6,
    mu = 0.5,
    num_customers = 3000,
    seed = 321,
)

```

5.3.6 Аналитика модели Росса

Для модели Росса среднее время до падения находится через систему линейных уравнений.

```
function ross_analytic_crash_time(
    N,
    S,
    repairers,
    mean_time_to_failure,
    mean_repair_time,
)
    a = N / mean_time_to_failure
    mu = 1 / mean_repair_time
    m = S + 1
    A = zeros(Float64, m, m)
    b = ones(Float64, m)

    for s in 0:S
        idx = s + 1
        repair_rate = min(S - s, repairers) * mu
        if s == 0
            A[idx, idx] = a + repair_rate
            A[idx, idx + 1] = -repair_rate
        elseif s == S
            A[idx, idx] = a
            A[idx, idx - 1] = -a
        else
            A[idx, idx] = a + repair_rate
            A[idx, idx - 1] = -a
            A[idx, idx + 1] = -repair_rate
        end
    end

    times = A \ b
    return times[S + 1]
end
```

5.3.7 Имитация модели Росса

В одном прогоне сравнивается время следующего отказа и время ближайшего окончания ремонта. При отказе без резерва фиксируется событие `crash`.

```
while true
    next_repair = isempty(completion_times) ? Inf : minimum(completion_times)

    if next_failure <= next_repair
        now = next_failure
        if spares == 0
            push_ross_event!(
                rows,
                now,
                "crash",
                N,
                spares,
                repair_queue,
                busy_repairers;
                crashed = true,
            )
            break
        end

        spares -= 1
        push_ross_event!(
            rows,
            now,
            "failure",
            N,
            spares,
            repair_queue,
            busy_repairers,
        )
    else
        now = next_repair
        busy_repairers -= 1
        spares += 1
        push_ross_event!(
```

```

        rows,
        now,
        "repair_finish",
        N,
        spares,
        repair_queue,
        busy_repairers,
    )
end
end

```

5.3.8 Запуск модели Росса

```

QueueingModels.run_ross_experiment(;
    N = 10,
    S = 3,
    repairers = 1,
    mean_time_to_failure = 100.0,
    mean_repair_time = 1.0,
    runs = 300,
    seed = 150,
)

```

5.3.9 Параметрическое исследование модели Росса

```

QueueingModels.run_ross_parameter_scan(;
    N_values = [5, 10, 15],
    S_values = [1, 3],
    repairer_values = [1, 2],
    mean_time_to_failure = 100.0,
    mean_repair_time = 1.0,
    runs = 20,
    seed = 500,
)

```

6 Модель M/M/c

6.1 Базовый прогон

Базовый сценарий был выполнен скриптом `scripts/mmc.jl`.

```

julia> include("scripts/mmc.jl")
Precompiling Distributions finished.

```

Рисунок 6: Запуск базового сценария M/M/c

В результате были сохранены таблицы `mmc_customers.csv`, `mmc_events.csv`, `mmc_summary.csv` и графики очереди, занятых каналов, распределения времени ожидания и сравнения аналитики с имитацией.

6.2 Таблица `mmc_customers.csv`

Столбцы таблицы:

Таблица 5: Описание столбцов таблицы `mmc_customers.csv`

Столбец	Описание
<code>id</code>	номер заявки
<code>arrival_time</code>	момент прибытия заявки
<code>service_start</code>	момент начала обслуживания
<code>departure_time</code>	момент выхода из системы
<code>wait_time</code>	время ожидания в очереди
<code>service_time</code>	длительность обслуживания

Столбец	Описание
<code>system_time</code>	полное время пребывания в системе
<code>server_id</code>	номер обслуживающего канала

Первые строки таблицы:

Таблица 6: Первые строки `mmc_customers.csv`

<code>id</code>	<code>arrival_time</code>	<code>service_start</code>	<code>departure_time</code>	<code>wait_time</code>	<code>service_time</code>	<code>system_time</code>	<code>server_id</code>
1	0.145	0.145	0.953	0.000	0.808	0.808	1
2	1.100	1.100	1.235	0.000	0.135	0.135	2
3	1.167	1.167	2.261	0.000	1.094	1.094	1
4	3.246	3.246	4.663	0.000	1.417	1.417	2
5	5.091	5.091	6.565	0.000	1.474	1.474	1

6.3 Таблица `mmc_events.csv`

Столбцы таблицы:

Таблица 7: Описание столбцов таблицы `mmc_events.csv`

Столбец	Описание
<code>time</code>	момент события
<code>event</code>	тип события: <code>arrival</code> , <code>service_start</code> , <code>departure</code>
<code>customer_id</code>	номер заявки, к которой относится событие
<code>queue_length</code>	длина очереди после обработки события
<code>busy_servers</code>	число занятых каналов после события
<code>system_size</code>	число заявок в системе после события

Первые строки таблицы:

Таблица 8: Первые строки `mmc_events.csv`

<code>time</code>	<code>event</code>	<code>customer_id</code>	<code>queue_length</code>	<code>busy_servers</code>	<code>system_size</code>
0.145	<code>arrival</code>	1	1	0	1
0.145	<code>service_start</code>	1	0	1	1
0.953	<code>departure</code>	1	0	0	0
1.100	<code>arrival</code>	2	1	0	1
1.100	<code>service_start</code>	2	0	1	1

6.4 Таблица `mmc_summary.csv`

Столбцы таблицы:

Таблица 9: Описание столбцов таблицы `mmc_summary.csv`

Столбец	Описание
<code>lambda</code>	интенсивность входящего потока
<code>mu</code>	интенсивность обслуживания одного канала
<code>c</code>	число каналов

Столбец	Описание
num_customers	число смоделированных заявок
seed	зерно генератора случайных чисел
rho	загрузка системы на один канал
analytic_wq, sim_wq	аналитическое и имитационное среднее ожидание
analytic_w, sim_w	аналитическое и имитационное время в системе
analytic_lq, sim_lq	аналитическое и имитационное число заявок в очереди
analytic_l, sim_l	аналитическое и имитационное число заявок в системе
analytic_pwait, sim_pwait	вероятность ожидания
sim_utilization	имитационная загрузка каналов

Строка таблицы, часть 1:

Таблица 10: Строка `mmc_summary.csv` : параметры и временные метрики

lambda	mu	c	num_customers	seed	rho	analytic_wq	sim_wq	analytic_w	sim_w
0.9	0.5	2	5000	123	0.9	8.526	8.345	10.526	10.313

Строка таблицы, часть 2:

Таблица 11: Строка `mmc_summary.csv` : метрики длины очереди и загрузки

analytic_lq	sim_lq	analytic_l	sim_l	analytic_pwait	sim_pwait	sim_utilization
7.674	7.622	9.474	9.420	0.853	0.856	0.899

6.5 Графики базового прогона M/M/c

На графике сравнения показаны аналитические и имитационные значения `Wq`, `W`, `Lq`, `L`. Расхождения небольшие: например, `analytic_wq = 8.5263`, а `sim_wq = 8.3449`.

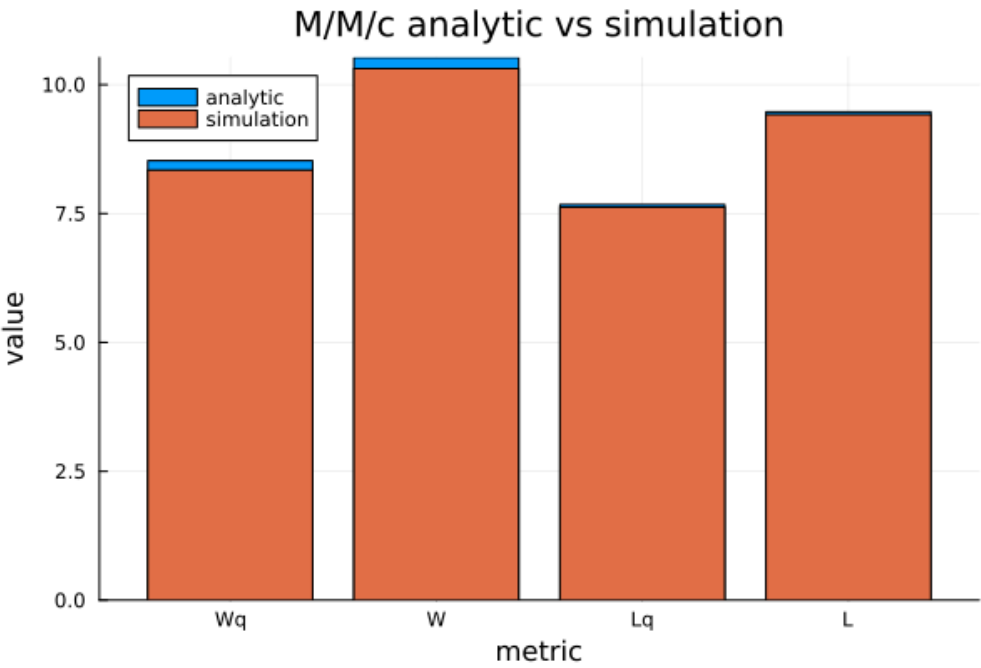


Рисунок 7: Сравнение аналитики и имитации M/M/c

График занятых каналов показывает, что при `rho = 0.9` два сервера часто работают одновременно.

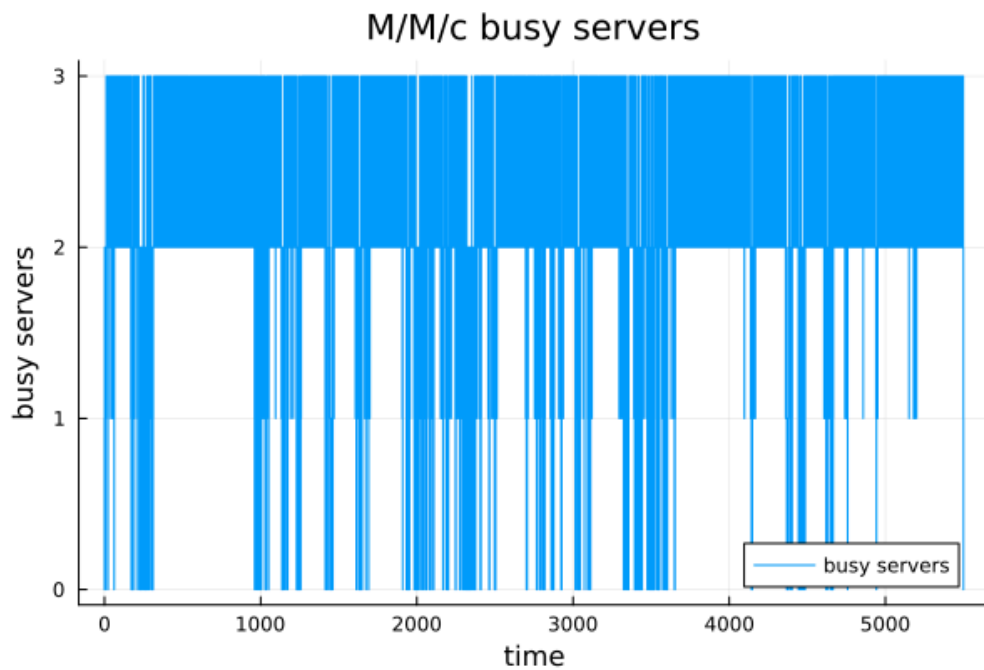


Рисунок 8: Число занятых каналов

График длины очереди демонстрирует сильные колебания: система устойчива, но из-за высокой загрузки очередь периодически заметно растёт.

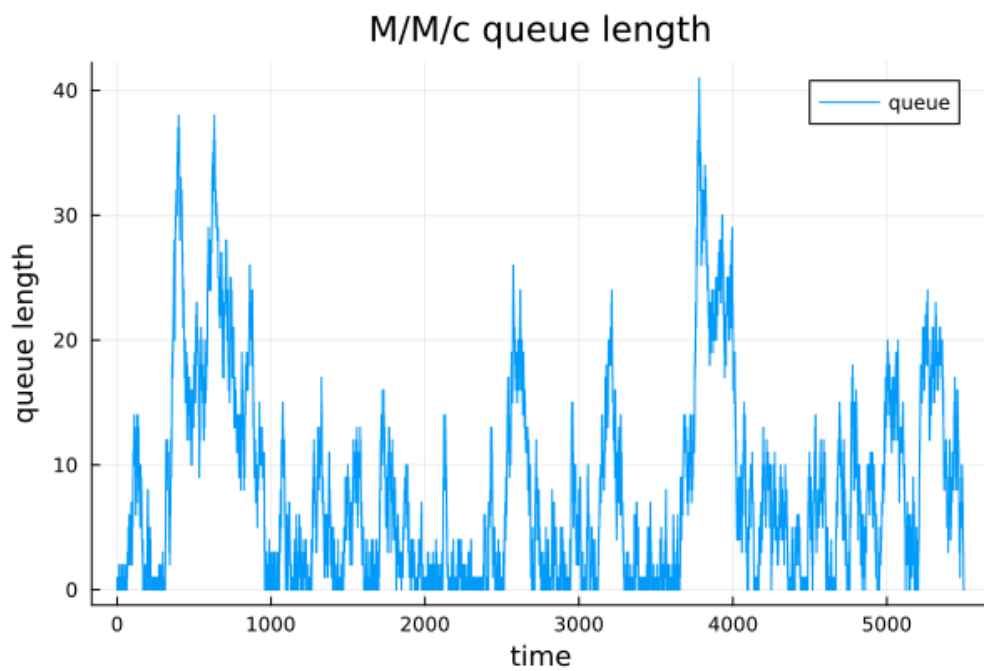


Рисунок 9: Длина очереди

Гистограмма времени ожидания показывает асимметричное распределение: много заявок ждут мало, но есть длинный правый хвост.

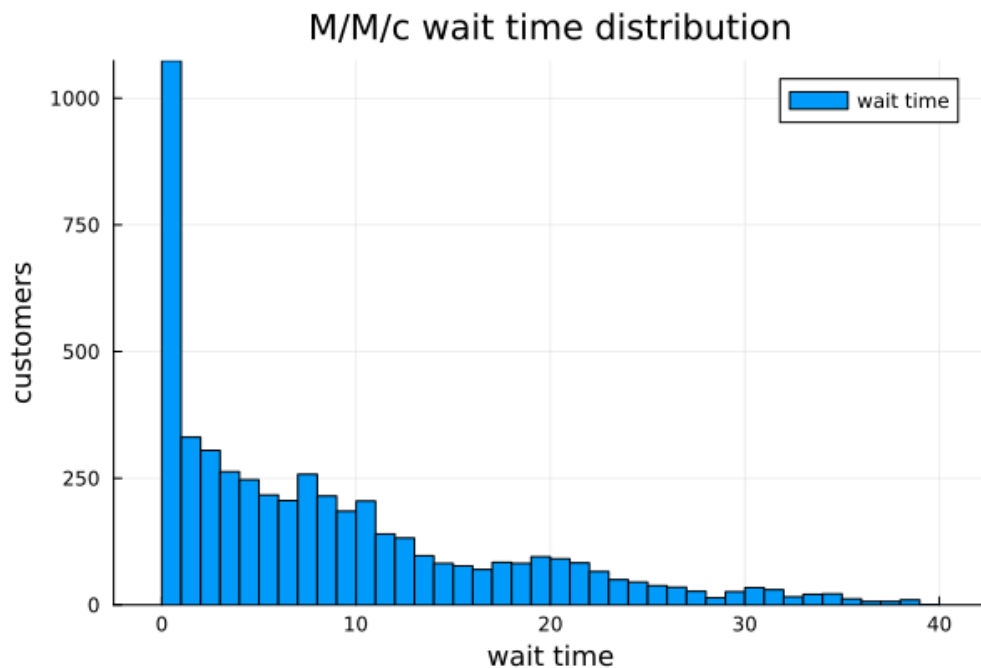


Рисунок 10: Гистограмма времени ожидания

После выполнения literate-скрипта были получены производные форматы `docs/mmc.md`, `scripts/mmc_clean.jl`, `notebooks/mmc.ipynb`.

```
julia> Literate.markdown("scripts/mmc_literate.jl", "docs";
    name = "mmc",
    credit = false)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/mmc.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/mmc.md"

julia> Literate.script("scripts/mmc_literate.jl", "scripts";
    name = "mmc_clean",
    credit = false)
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_clean.jl"

julia> Literate.notebook("scripts/mmc_literate.jl", "notebooks";
    name = "mmc",
    execute = true,
    credit = false)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_literate.jl`
[ Info: executing notebook `mmc.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/mmc.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/mmc.ipynb"
```

Рисунок 11: Производные форматы M/M/c

6.6 Параметрическое исследование M/M/c

Параметрическое исследование было выполнено скриптом `scripts/mmc_parameters.jl`.

```
julia> include("scripts/mmc_parameters.jl")
Could not create decoration from factory! Running with no decorations.
M/M/c parameters scan completed
```

Рисунок 12: Запуск параметрического исследования M/M/c

6.7 Таблица mmc_parameter_scan.csv

Столбцы таблицы:

Таблица 12: Описание столбцов таблицы mmc_parameter_scan.csv

Столбец	Описание
lambda	интенсивность входящего потока
mu	интенсивность обслуживания
c	число каналов обслуживания
rho	загрузка одного канала
analytic_wq	аналитическое среднее ожидание в очереди
sim_wq	имитационное среднее ожидание
analytic_w	аналитическое время в системе
sim_w	имитационное время в системе
sim_utilization	имитационная загрузка каналов

Первые строки таблицы:

Таблица 13: Первые строки mmc_parameter_scan.csv

lambda	mu	c	rho	analytic_wq	sim_wq	analytic_w	sim_w	sim_utilization
0.3	0.5	1	0.60	3.000	2.819	5.000	4.825	0.597
0.3	0.5	2	0.30	0.198	0.224	2.198	2.240	0.301
0.3	0.5	3	0.20	0.021	0.028	2.021	2.042	0.204
0.3	0.5	4	0.15	0.002	0.005	2.002	1.936	0.149
0.3	0.5	5	0.12	0.000	0.000	2.000	1.998	0.121

Тепловая карта загрузки показывает, что загрузка растёт при увеличении lambda и уменьшается при увеличении числа каналов.

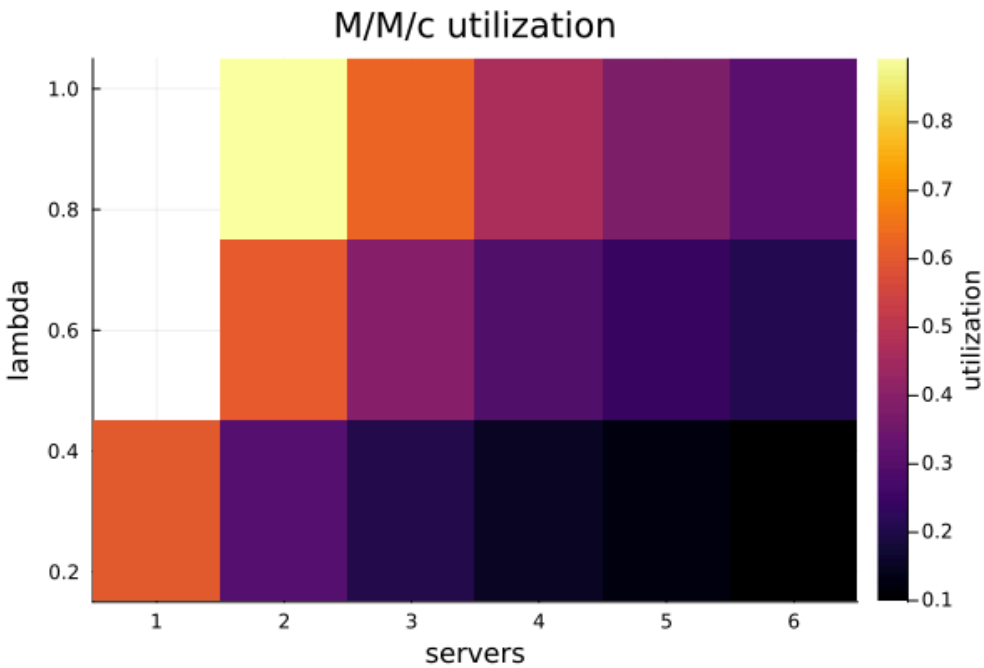


Рисунок 13: Тепловая карта загрузки M/M/c

График зависимости ожидания от числа каналов показывает, что добавление серверов резко снижает Wq, особенно при больших значениях lambda.

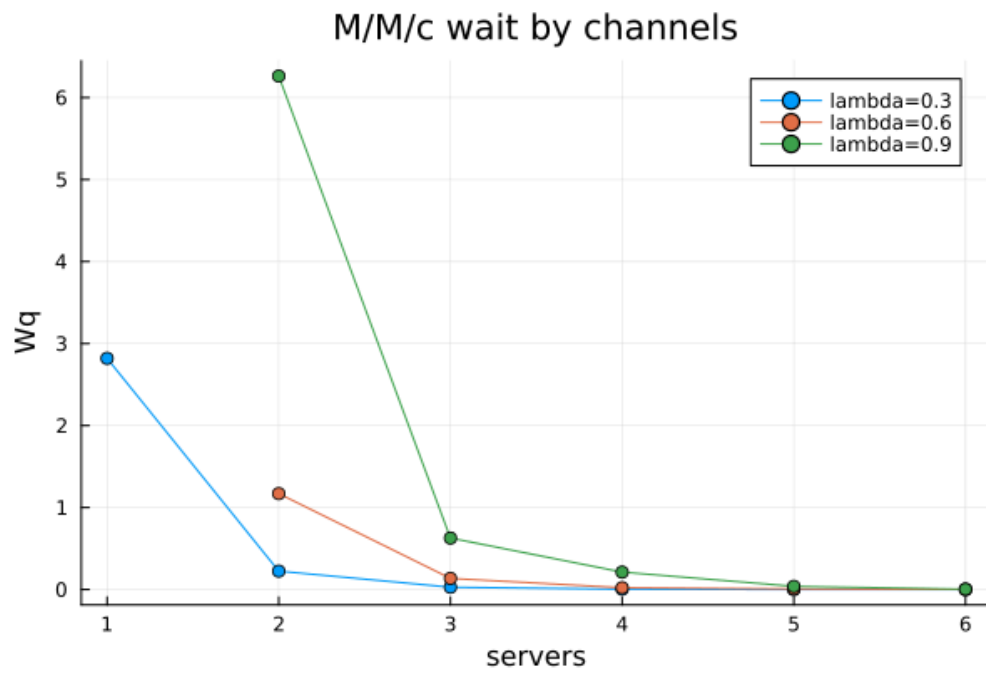


Рисунок 14: Ожидание в зависимости от числа каналов

График зависимости ожидания от интенсивности входящего потока показывает рост очереди при увеличении `lambda`.

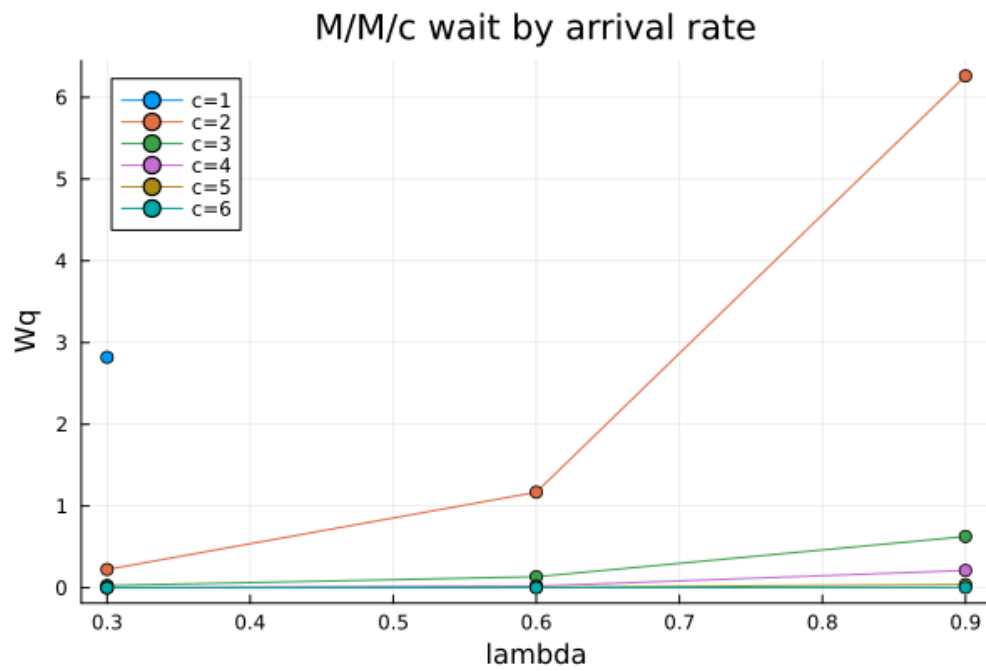


Рисунок 15: Ожидание в зависимости от lambda

После выполнения `literate`-версии параметрического сценария были получены [docs/mmc_parameters.md](#), [scripts/mmc_parameters_clean.jl](#), [notebooks/mmc_parameters.ipynb](#).

```
julia> Literate.markdown("scripts/mmc_parameters_literate.jl", "docs";
    name = "mmc_parameters",
    credit = false)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_parameters_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/mmc_parameters.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/mmc_parameters.md"

julia> Literate.script("scripts/mmc_parameters_literate.jl", "scripts";
    name = "mmc_parameters_clean",
    credit = false)
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_parameters_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_parameters_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_parameters_clean.jl"

julia> Literate.notebook("scripts/mmc_parameters_literate.jl", "notebooks";
    name = "mmc_parameters",
    execute = true,
    credit = false)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/mmc_parameters_literate.jl`
[ Info: executing notebook `mmc_parameters.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/mmc_parameters.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/mmc_parameters.ipynb"
```

Рисунок 16: Производные форматы параметрического исследования M/M/c

7 Модель Росса

7.1 Базовый прогон

Базовый сценарий модели Росса был выполнен скриптом `scripts/ross.jl`.

```
julia> include("scripts/ross.jl")
Could not create decoration from factory! Running with no decorations.
Ross model experiment completed.
```

Рисунок 17: Запуск модели Росса

7.2 Таблица `ross_events_sample.csv`

Столбцы таблицы:

Таблица 14: Описание столбцов таблицы `ross_events_sample.csv`

Столбец	Описание
<code>time</code>	момент события
<code>event</code>	тип события: <code>start</code> , <code>failure</code> , <code>repair_start</code> , <code>repair_finish</code> , <code>crash</code>
<code>working</code>	число работающих машин
<code>spares</code>	число резервных машин
<code>broken</code>	число сломанных машин
<code>repair_queue</code>	число машин в очереди на ремонт
<code>busy_repairers</code>	число занятых ремонтников
<code>good_machines</code>	число исправных машин

Первые строки таблицы:

Таблица 15: Первые строки `ross_events_sample.csv`

time	event	working	spares	broken	repair_queue	busy_repairers	good_machines
0.000	start	10	3	0	0	0	13
1.947	failure	10	2	0	0	0	12
1.947	repair_start	10	2	1	0	1	12
4.396	repair_finish	10	3	0	0	0	13
12.247	failure	10	2	0	0	0	12

7.3 Таблица `ross_runs.csv`

Столбцы таблицы:

Таблица 16: Описание столбцов таблицы `ross_runs.csv`

Столбец	Описание
run_id	номер прогона
crash_time	время до падения системы
mean_repair_queue	средняя длина очереди на ремонт
repairer_utilization	загрузка ремонтника

Первые строки таблицы:

Таблица 17: Первые строки `ross_runs.csv`

run_id	crash_time	mean_repair_queue	repairer_utilization
1	1669.888	0.0123	0.1063
2	42675.710	0.0101	0.1011
3	18234.460	0.0112	0.1047
4	2719.418	0.0136	0.1004
5	3409.050	0.0084	0.1056

7.4 Таблица `ross_summary.csv`

Столбцы таблицы:

Таблица 18: Описание столбцов таблицы `ross_summary.csv`

Столбец	Описание
N	число рабочих машин
S	число резервных машин
repairers	число ремонтников
runs	число прогонов
mean_crash_time	среднее время до падения
std_crash_time	стандартное отклонение времени до падения
analytic_crash_time	аналитическая оценка времени до падения
mean_repair_queue	средняя очередь на ремонт
repairer_utilization	средняя загрузка ремонтников

Строка таблицы:

Таблица 19: Строка ross_summary.csv

N	S	repairers	runs	mean_crash_time	std_crash_time	analytic_crash_time	mean_repair_queue	repairer_utilization
10	3	1	300	11996.189	12450.172	12340.000	0.0124	0.1017

7.5 Графики базового прогона модели Росса

Гистограмма времени до падения показывает высокую вариативность случайного времени отказа системы.

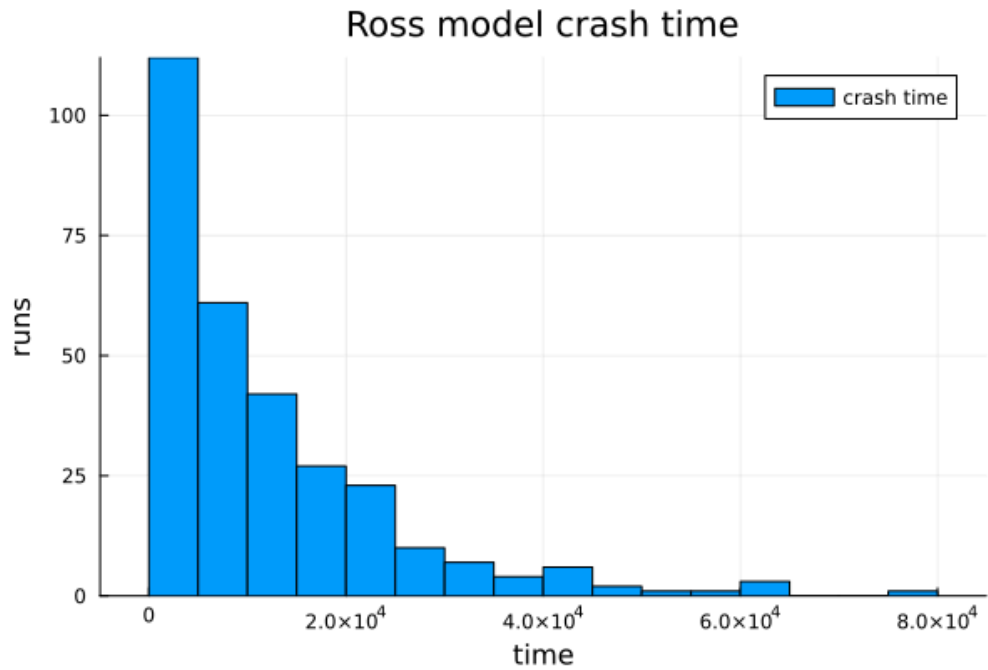


Рисунок 18: Гистограмма времени до падения

График числа исправных машин показывает ступенчатую событийную динамику отказов и ремонтов.

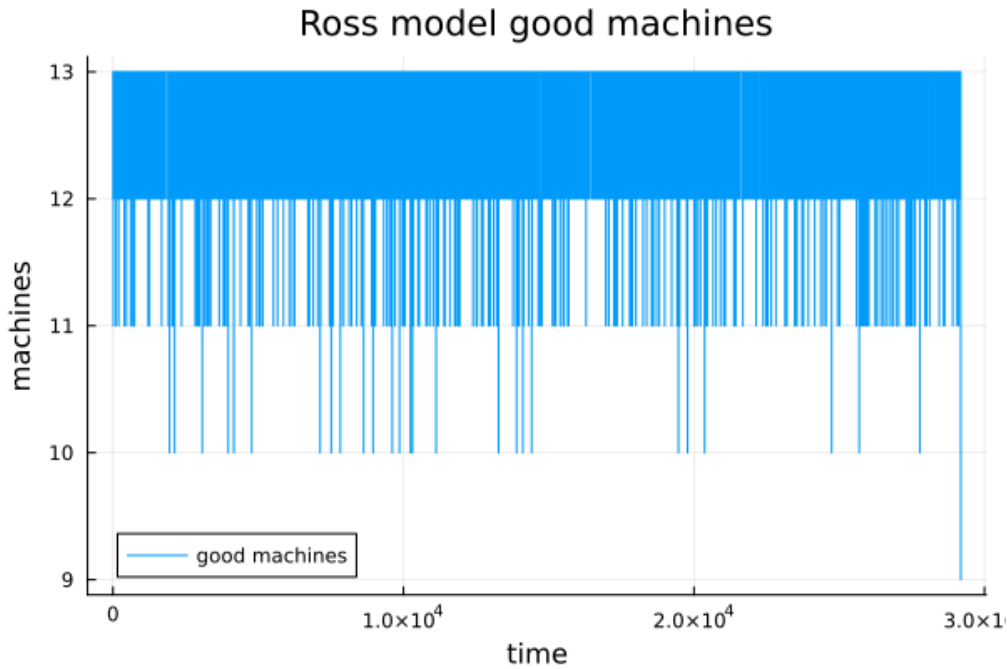


Рисунок 19: Число исправных машин

График загрузки ремонтника показывает, что при базовых параметрах ремонтник занят примерно на 0.1017 доли времени.

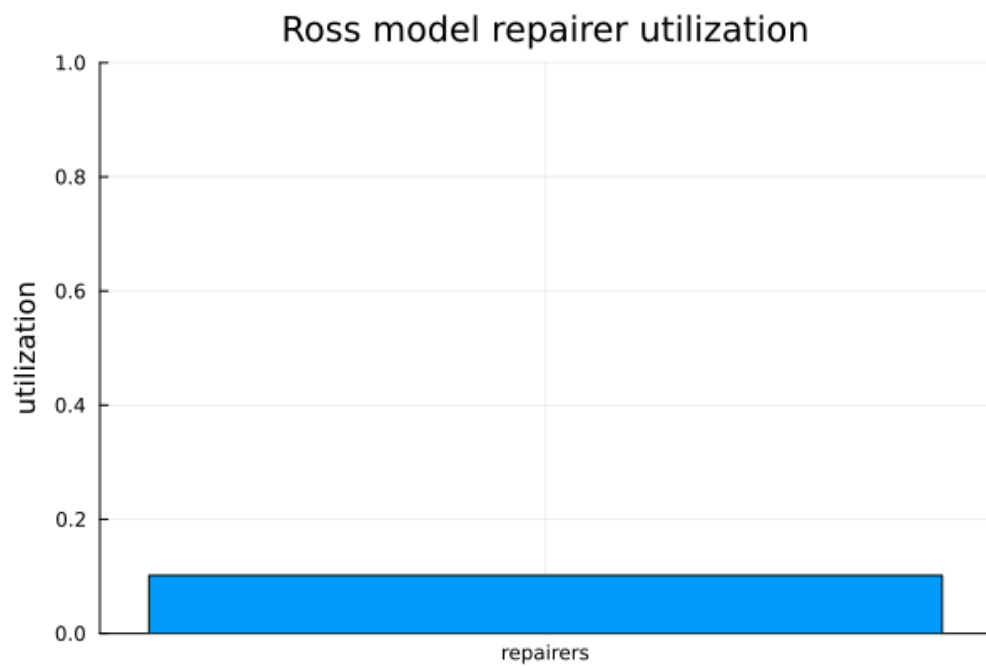


Рисунок 20: Загрузка ремонтника

График очереди на ремонт показывает, что очередь почти всегда мала; среднее значение по `ross_summary.csv` равно `0.01238`.

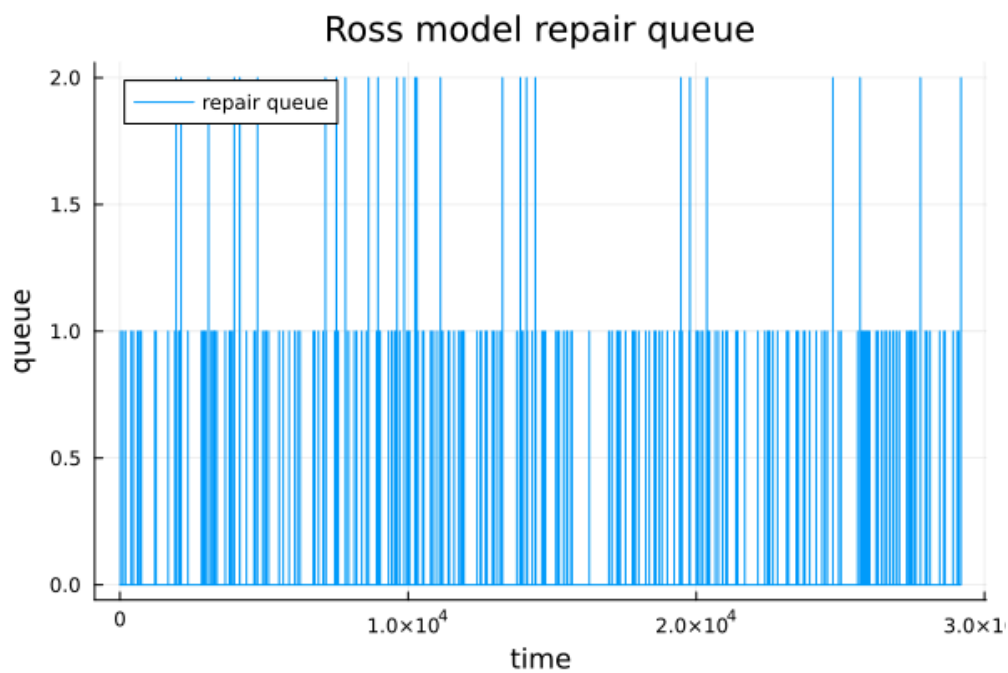


Рисунок 21: Очередь на ремонт

Сравнение имитации и аналитики показывает близкие значения: среднее по имитации `11996.19`, аналитическая оценка `12340.00`.

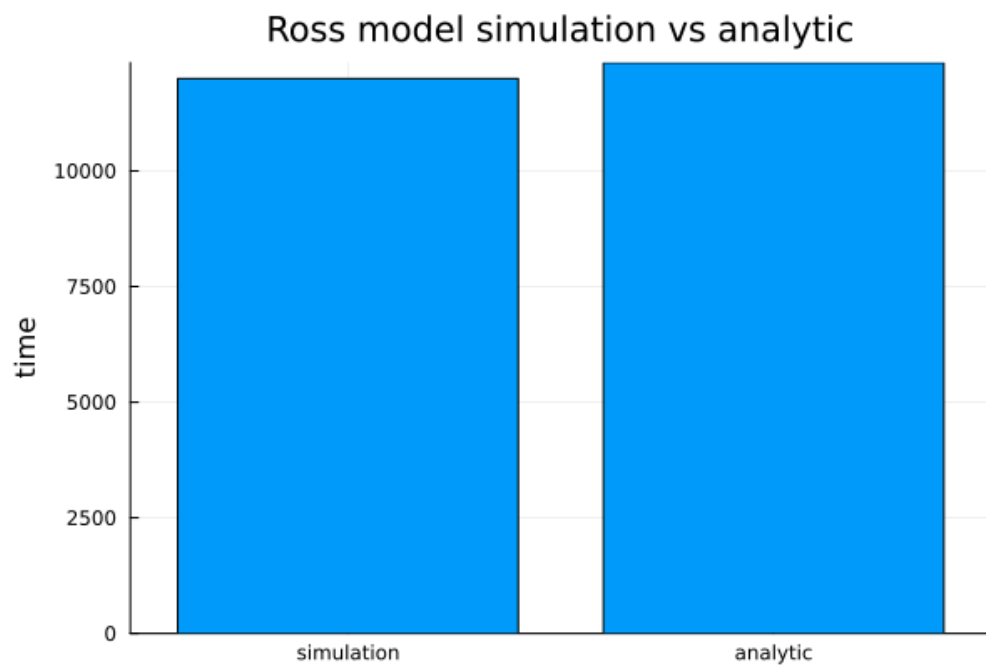


Рисунок 22: Сравнение имитации и аналитики

График числа резервных машин показывает, как запас уменьшается при отказах и восстанавливается после ремонта.

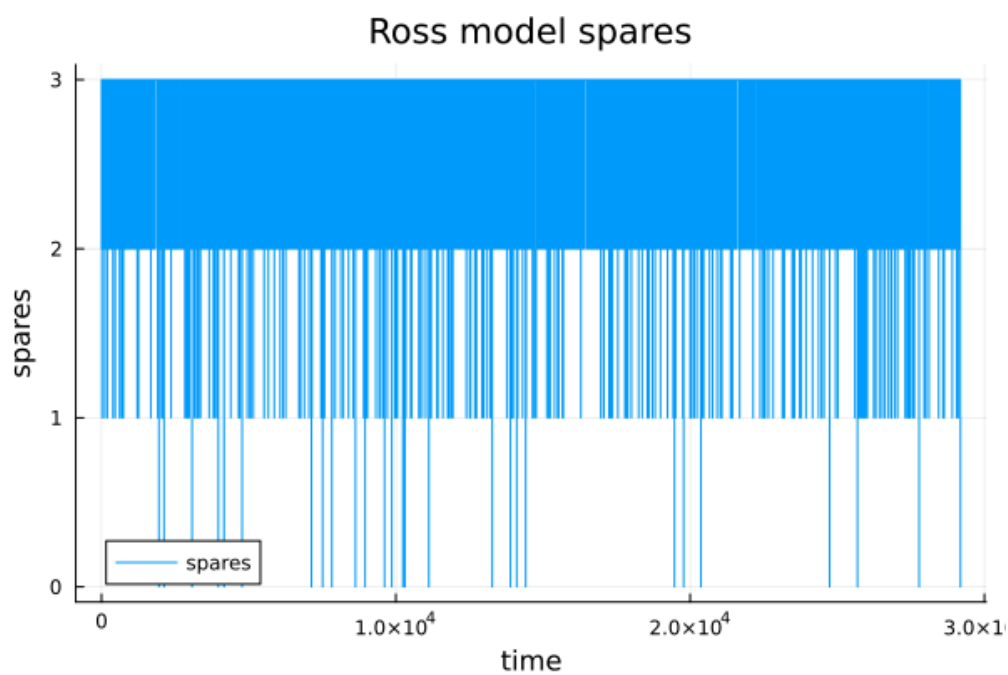


Рисунок 23: Число резервных машин

После выполнения `literate`-версии базового сценария были получены [docs/ross.md](#), [scripts/ross_clean.jl](#), [notebooks/ross.ipynb](#).

```
julia> Literate.markdown("scripts/ross_literate.jl", "docs";
    name = "ross",
    credit = false)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross.md"

julia> Literate.script("scripts/ross_literate.jl", "scripts";
    name = "ross_clean",
    credit = false)
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_clean.jl"

julia> Literate.notebook("scripts/ross_literate.jl", "notebooks";
    name = "ross",
    execute = true,
    credit = false)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_literate.jl`
[ Info: executing notebook `ross.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/ross.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/ross.ipynb"
```

Рисунок 24: Производные форматы модели Росса

7.6 Параметрическое исследование модели Росса

Параметрический сценарий был выполнен скриптом `scripts/ross_parameters.jl`.

```
julia> include("scripts/ross_parameters.jl")
Could not create decoration from factory! Running with no decorations.
Ross parameter scan completed.
```

Рисунок 25: Запуск параметрического исследования Росса

7.7 Таблица `ross_parameter_scan.csv`

Столбцы таблицы:

Таблица 20: Описание столбцов таблицы `ross_parameter_scan.csv`

Столбец	Описание
<code>N</code>	число рабочих машин
<code>S</code>	число резервных машин
<code>repairers</code>	число ремонтников
<code>runs</code>	число прогонов для сценария
<code>mean_crash_time</code>	среднее время до падения
<code>std_crash_time</code>	стандартное отклонение
<code>analytic_crash_time</code>	аналитическая оценка
<code>mean_repair_queue</code>	средняя очередь на ремонт
<code>repairer_utilization</code>	загрузка ремонтников

Первые строки таблицы:

Таблица 21: Первые строки `ross_parameter_scan.csv`

N	S	repairers	runs	mean_crash_time	std_crash_time	analytic_crash_time	mean_repair_queue	repairer_utilization
5	1	1	20	432.737	396.997	440.000	0.0000	0.0501
5	1	2	20	294.977	225.544	440.000	0.0000	0.0249
5	3	1	20	224341.051	181764.863	177280.000	0.0028	0.0503
5	3	2	20	687686.634	791879.395	690080.000	0.0000	0.0251
10	1	1	20	147.400	152.540	120.000	0.0000	0.0909

График зависимости времени до падения от N показывает, что при росте числа рабочих машин суммарная интенсивность отказов увеличивается, поэтому время до падения уменьшается.

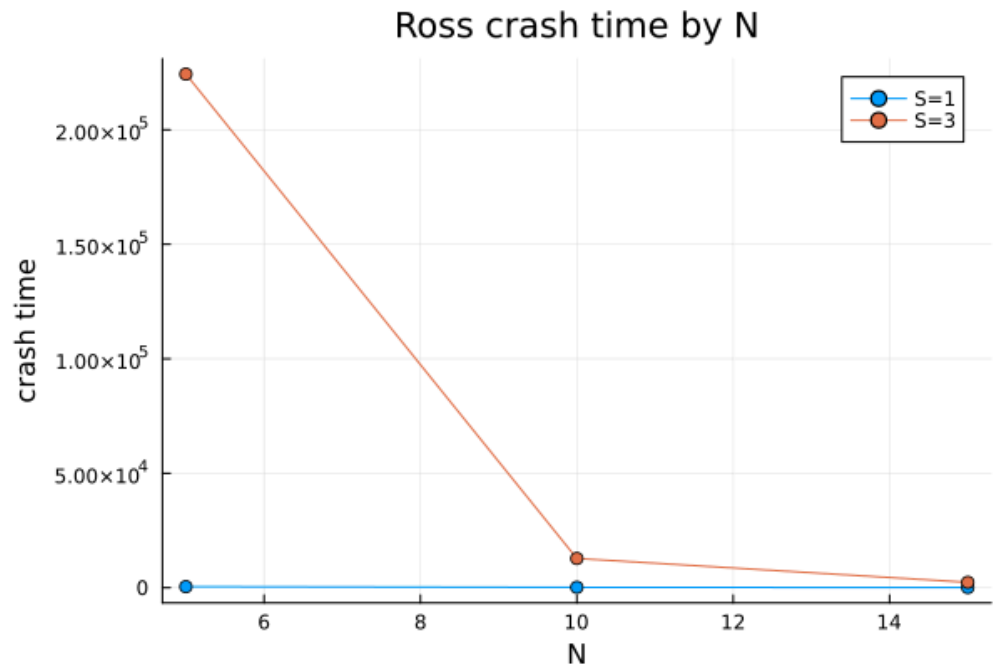


Рисунок 26: Время до падения в зависимости от N

График зависимости времени до падения от числа резервных машин показывает сильный положительный эффект резервирования.

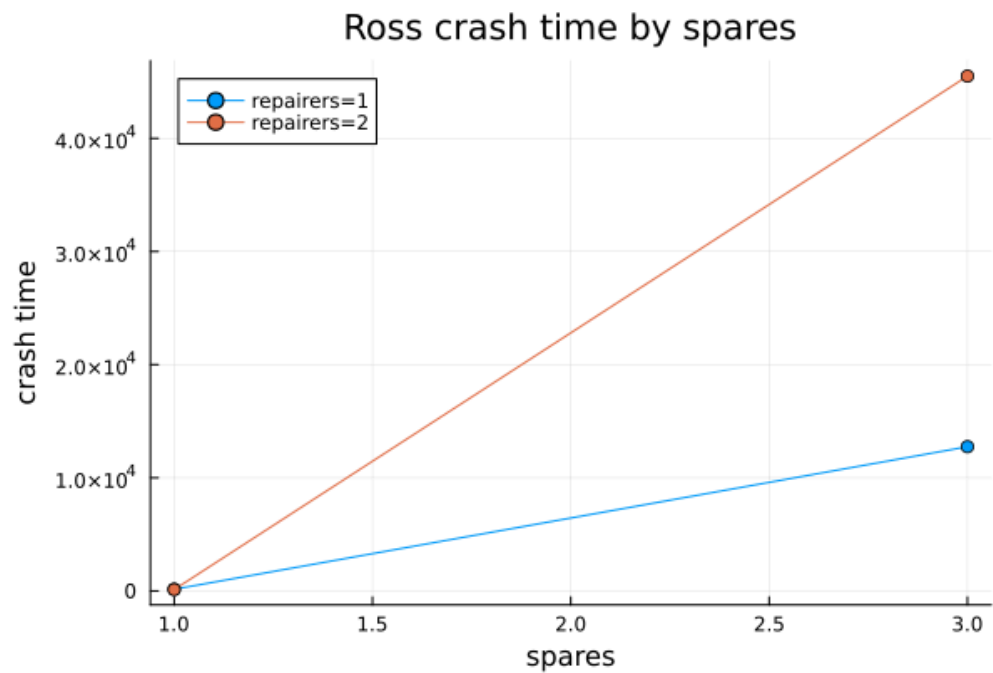


Рисунок 27: Время до падения в зависимости от S

График сравнения числа ремонтников показывает, что увеличение числа ремонтников повышает устойчивость системы, особенно при большем запасе резервных машин.

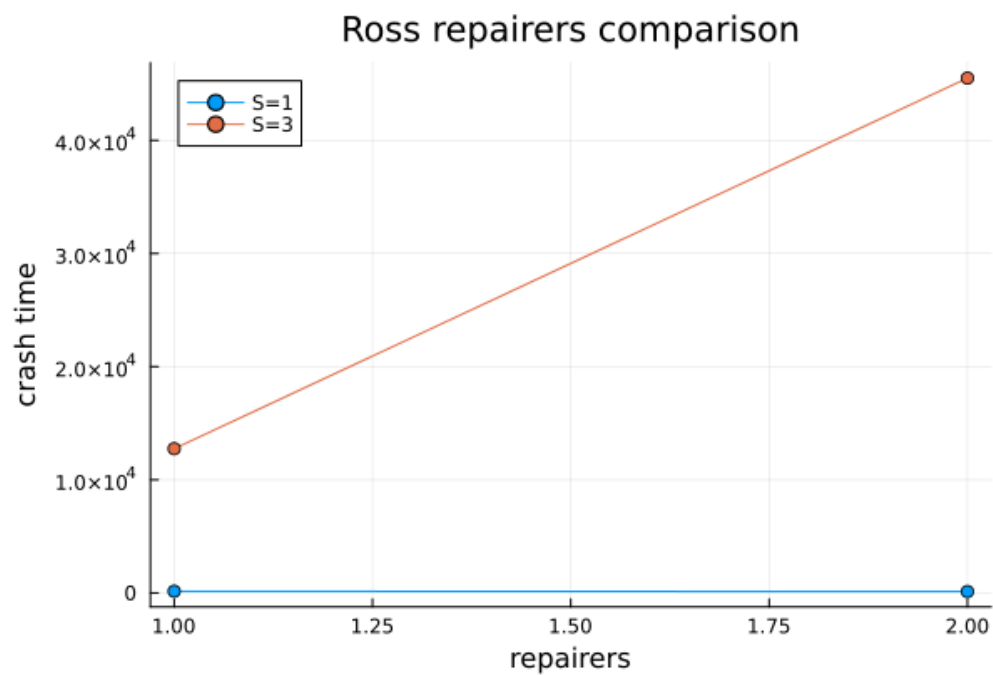


Рисунок 28: Сравнение числа ремонтников

График загрузки ремонтников по сценариям показывает, что загрузка снижается при добавлении ремонтника и зависит от числа рабочих и резервных машин.

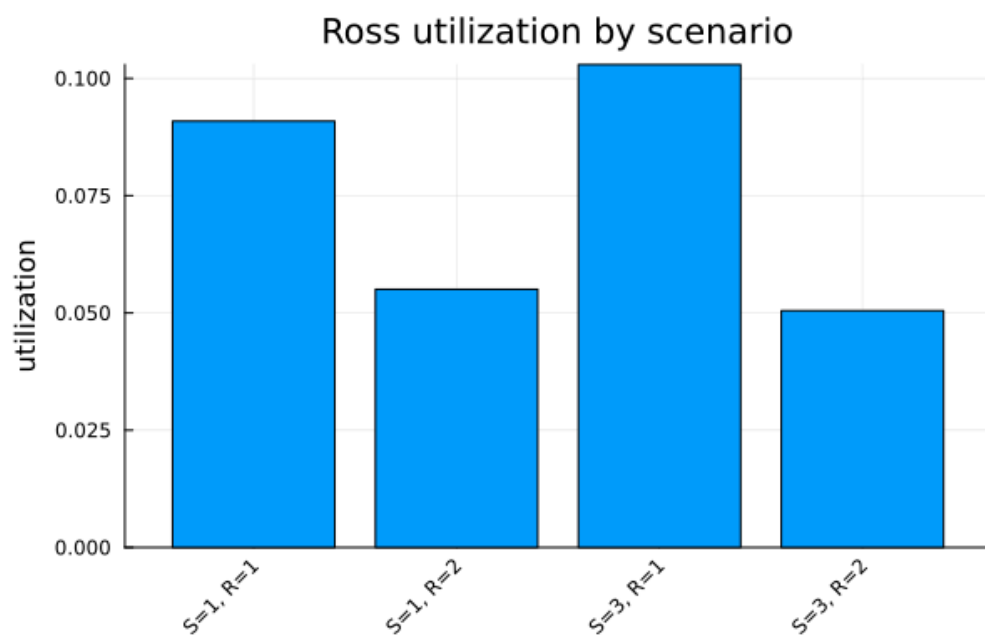


Рисунок 29: Загрузка ремонтников по сценариям

После выполнения literate-версии параметрического сценария были получены [docs/ross_parameters.md](#), [scripts/ross_parameters_clean.jl](#), [notebooks/ross_parameters.ipynb](#).

```

julia> Literate.markdown("scripts/ross_parameters_literate.jl", "docs";
    name = "ross_parameters",
    credit = false)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_parameters_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross_parameters.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross_parameters.md"

julia> Literate.markdown("scripts/ross_parameters_literate.jl", "docs";
    name = "ross_parameters",
    credit = false)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_parameters_literate.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross_parameters.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/docs/ross_parameters.md"

julia> Literate.notebook("scripts/ross_parameters_literate.jl", "notebooks";
    name = "ross_parameters",
    execute = true,
    credit = false)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/scripts/ross_parameters_literate.jl`
[ Info: executing notebook `ross_parameters.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/ross_parameters.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab07/lab_07_des/notebooks/ross_parameters.ipynb"

```

Рисунок 30: Производные форматы параметрического исследования Росса

8 Производные форматы

Для каждого из четырёх сценариев были подготовлены literate-скрипты:

- `mnc_literate.jl`;
- `mnc_parameters_literate.jl`;
- `ross_literate.jl`;
- `ross_parameters_literate.jl`.

Для каждого сценария через `Literate.jl` были получены три формата:

- Markdown-документ в `docs/`;
- чистый Julia-скрипт в `scripts/`;
- выполненный notebook в `notebooks/`.

Отдельный запуск `clean`-версий не выполнялся: результаты моделей были получены обычными скриптами, а `clean`-версии использовались как производный формат.

9 Выводы

В ходе работы был создан проект `lab_07_des` в структуре `DrWatson`, реализован модуль `QueueingModels.jl`, выполнены базовые и параметрические сценарии для `M/M/c` и модели Росса.

Для `M/M/c` получено, что при высокой загрузке `rho = 0.9` очередь заметно растёт, но имитационные оценки близки к аналитическим: `sim_wq = 8.3449` против `analytic_wq = 8.5263`. Параметрическое исследование показало, что увеличение числа каналов снижает ожидание, а рост `lambda` увеличивает загрузку и очередь.

Для модели Росса получено среднее время до падения `11996.19` при аналитической оценке `12340.00`. Параметрическое исследование показало, что увеличение числа резервных машин существенно повышает время до падения, а увеличение числа ремонтников снижает загрузку ремонтной подсистемы и повышает устойчивость системы.

Все результаты сохранены в CSV-таблицы, построены графики, подготовлены literate-представления и производные форматы `md`, `ipynb`, `clean`.

Список литературы

1. Law A. M. [Simulation Modeling and Analysis](#). 5-е изд. McGraw-Hill Education, 2014.
 2. Harchol-Balter M. [Performance Modeling and Design of Computer Systems: Queueing Theory in Action](#). Cambridge University Press, 2013.
 3. Ross S. M. [Simulation](#). 5-е изд. Academic Press, 2013.
 4. Bezanson J. и др. [Julia: A Fresh Approach to Numerical Computing](#) // SIAM Review. 2017. Т. 59, № 1. С. 65–98.
 5. Datseris G. и др. [DrWatson: The perfect sidekick for your scientific inquiries](#) // Journal of Open Source Software. 2020. Т. 5, № 54. С. 2673.
 6. Knuth D. E. [Literate Programming](#) // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.
 7. Ekström F., contributors. [Literate.jl](#). 2026.
-

Повторное использование

[CC BY 4.0](#)